

Security Assessment of an Unknown System in an Isolated Virtual Network

Course:	Network Security
Section:	A
Group No.:	
Target VM Used:	DVWA (Damn Vulnerable Web Application)
NAT Network Used:	MT_Project

Group Members & Assessment

SL	Student ID	Student Name	Implementation (10)	VIVA (10)	Total (20)
1	23-54755-3	Tasnim Tiba			
2	23-54780-3	Md. Emran Hossain			
3	23-54785-3	Muhammad Abdur Rahaman Tashrif			

Contents

1. Introduction	3
2. Target System Description and Setup Evidence	3
2.1 Name and Source of the Selected System.....	3
2.2 Reason for Choosing This System	3
2.3 Target VM Evidence.....	3
3. Network Setup Evidence	3
3.1 Isolated NAT Network Configuration	4
3.2 IP Addressing	4
4. System Discovery Results	5
4.1 Port and Service Scan	6
4.2 Interpretation of Findings	6
5. Network Communication Observation	6
5.1 Captured Traffic	6
5.2 Sensitive Information Exposed	7
6. Authentication Strength Evaluation	8
6.1 Extracting Account and Hash Information	8
6.2 Password Recovery.....	9
6.3 Why the Passwords Were Weak	9
7. Security Analysis and Recommendations.....	9
7.1 Identified Weaknesses	9
7.2 Why These Weaknesses Exist	10
7.3 Proposed Countermeasures.....	10
8. Task Distribution Among Group Members	10
9. Conclusion	10

1. Introduction

This report documents a security assessment carried out against a deliberately vulnerable web application, DVWA (Damn Vulnerable Web Application), deployed inside an isolated VirtualBox NAT network. The purpose of the exercise was to practise the full audit workflow taught in this course: environment isolation, blind service discovery, traffic analysis, offline credential auditing, and the translation of technical findings into risk-based recommendations. No system outside the isolated lab network was accessed or scanned at any point.

2. Target System Description and Setup Evidence

2.1 Name and Source of the Selected System

The selected target is DVWA (Damn Vulnerable Web Application), an open-source PHP/MySQL web application distributed at <https://github.com/digininja/DVWA> and maintained for security training. It has adjustable difficulty levels (Low / Medium / High / Impossible).

2.2 Reason for Choosing This System

DVWA was chosen because it is a web-facing application rather than a general-purpose OS image, which lets the assessment demonstrate web-layer discovery and traffic analysis in addition to the host- and service-level techniques covered by OS-based targets such as Metasploitable2. For this project, DVWA was deployed on a dedicated Ubuntu Server virtual machine inside Oracle VirtualBox, running the official `vulnerables/web-dvwa` Docker image to host the application with a pre-configured Apache, PHP, and MySQL stack. This VM was placed on its own isolated NAT Network, `MT_Project`, alongside the Kali Linux attacker machine, keeping the target fully separate from the attacking system as required by the assignment. DVWA's vulnerability set maps directly onto the weaknesses this course asks us to identify: unauthenticated/weak login credentials, plaintext transport over HTTP, and crackable password hashes stored in its MySQL database.

2.3 Target VM Evidence

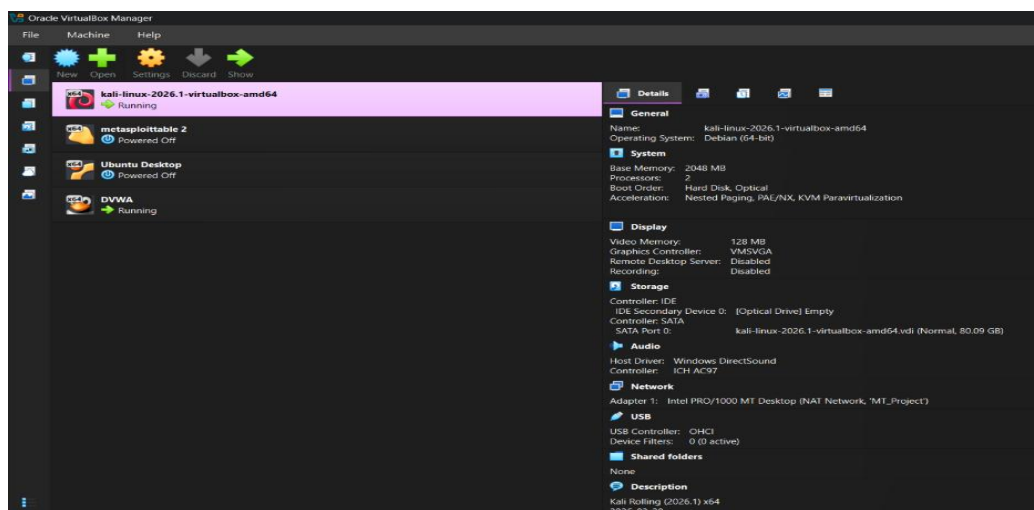


Figure 01: Kali Linux and DVWA inside Virtual Machine

The screenshot above shows the DVWA target VM in a running state, reachable at its assigned NAT-network IP address. The login banner confirms the application name and version, which was later cross-checked against known CVEs for that release during the discovery phase.

3. Network Setup Evidence

3.1 Isolated NAT Network Configuration

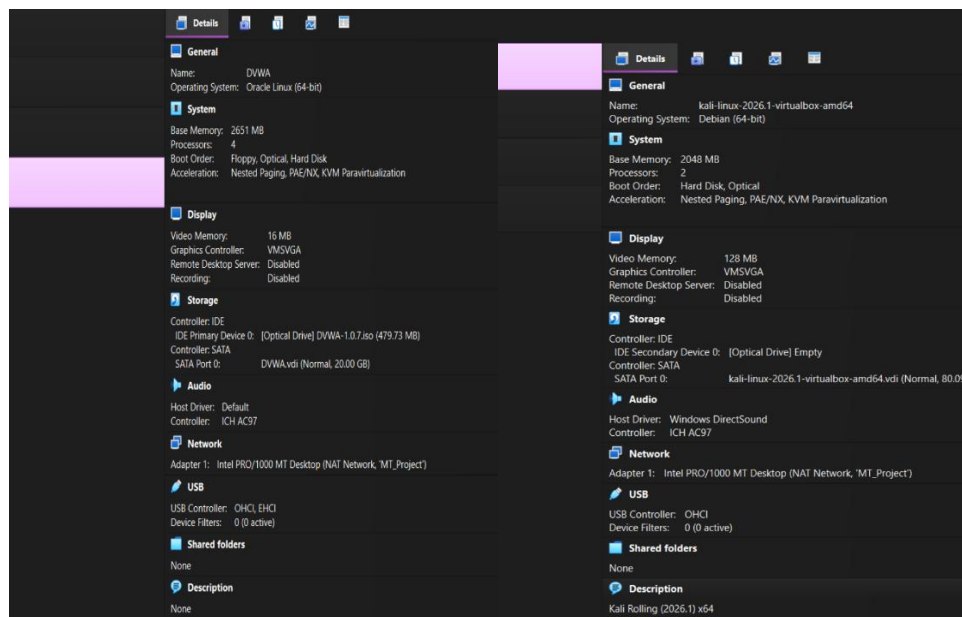


Figure 02: Kali Linux in NAT Network Configuration Figure 03: DVWA in NAT Network Configuration

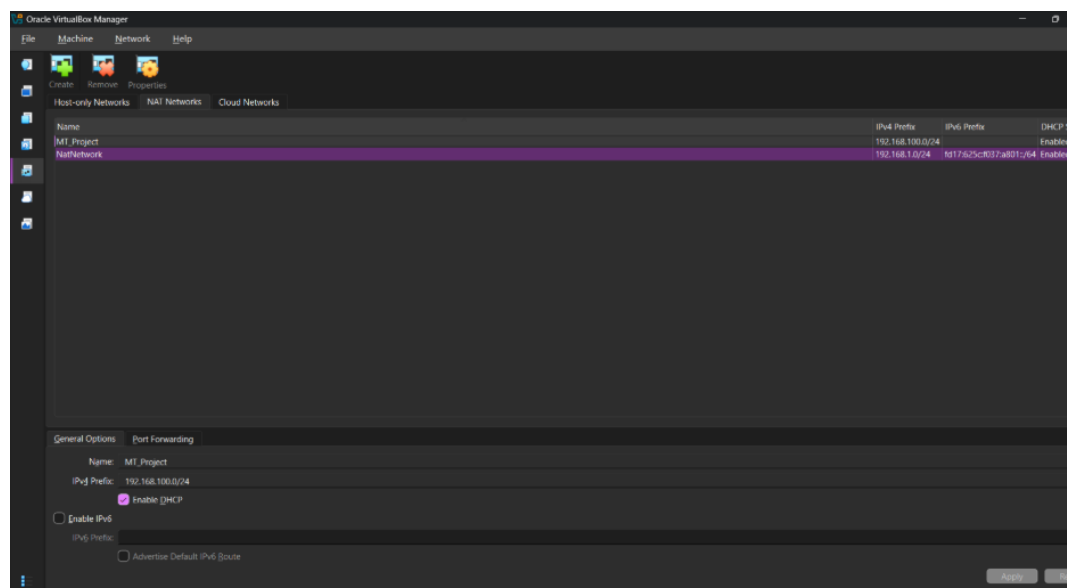


Figure 04: A dedicated NAT Network Configuration (MT_Project)

A dedicated NAT Network named MT_Project was created in VirtualBox, distinct from the network used in the weekly labs, so that traffic generated during this assessment could not reach or be reached by any other lab exercise or the host's regular network. Both the Kali attacker VM and the DVWA target VM (Ubuntu Server running the DVWA Docker container) were attached exclusively to MT_Project, with 'Enable Network' active and no bridged or host-only adapters in use, ensuring the two machines could only route to each other.

3.2 IP Addressing

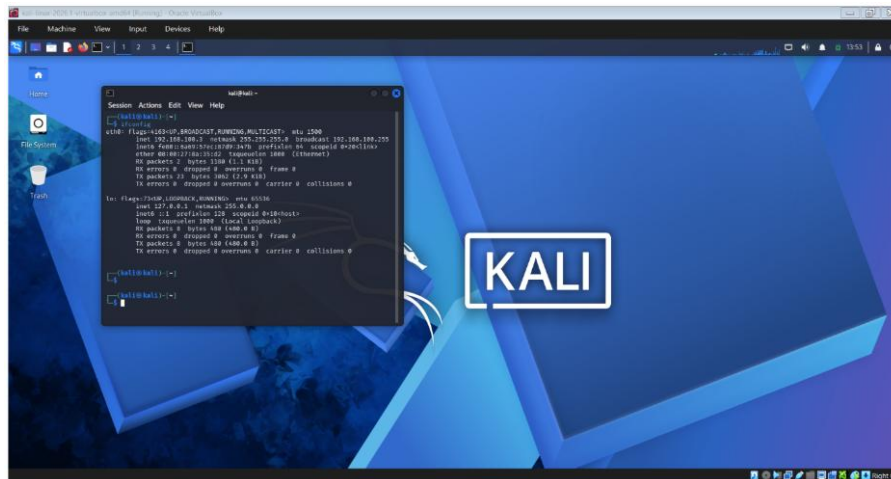


Figure 05: IP Address of Kali Linux

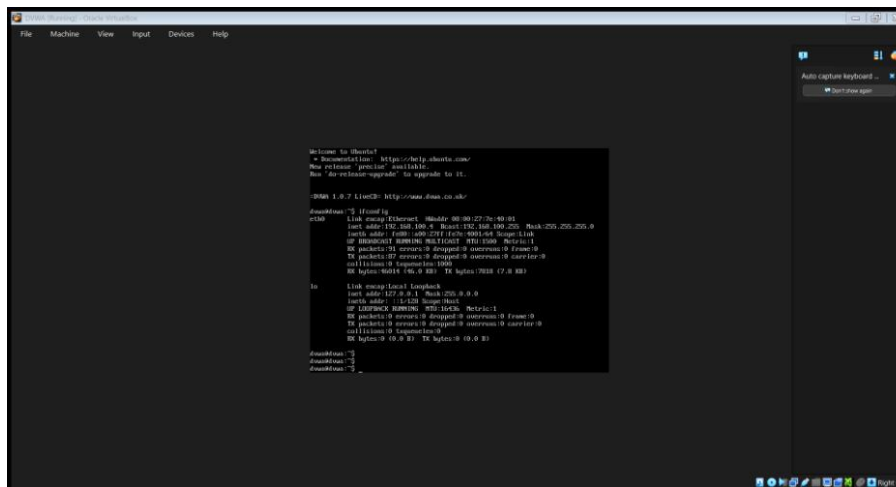


Figure 06: IP Address of DVWA on Ubuntu

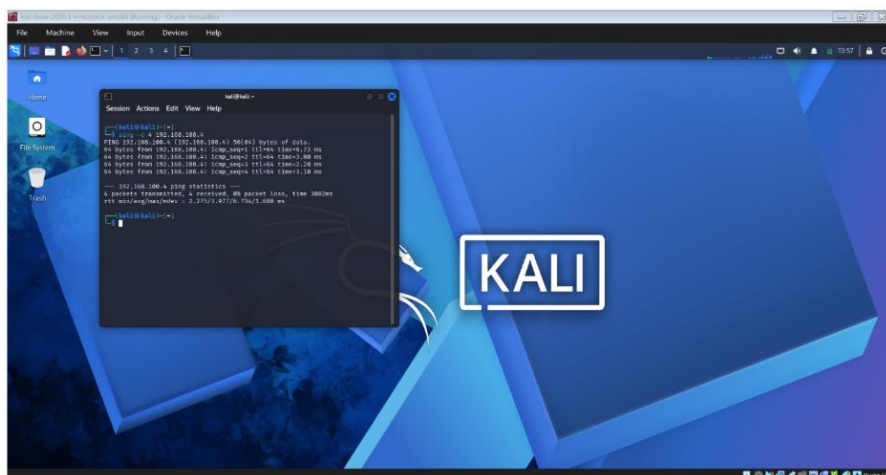


Figure 07: Connection evidence of Kali & DVWA using ping

Kali Linux was assigned 192.168.100.3 and the DVWA target was assigned 192.168.100.4 on the isolated NAT network. Reachability was confirmed with a simple ping between the two hosts before any scanning began, verifying Phase 1's requirement that the systems could communicate before discovery started.

4. System Discovery Results

4.1 Port and Service Scan

```
kali@kali: ~  
Session Actions Edit View Help  
— 192.168.100.4 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3082ms  
rtt min/avg/max/mdev = 2.275/3.977/6.734/1.680 ms  
  
kali@kali: ~  
$ nmap -sV 192.168.100.4  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-26 13:57 -0400  
Nmap scan report for 192.168.100.4  
Host is up (0.0015s latency).  
Not shown: 995 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
21/tcp    open  ftp      ProFTPD 1.3.2c  
22/tcp    open  ssh      OpenSSH 5.3p1 Debian 3ubuntu4 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http     Apache httpd 2.2.14 ((Unix) DAV/2 mod_ssl/2.2.14 OpenSSL/0.9.8l PHP/5.3.1 mod_apreq2-20090110/2.7.1 mod_perl/2.0.4 Perl/v5.10.1)  
443/tcp   open  ssl/http Apache httpd 2.2.14 ((Unix) DAV/2 mod_ssl/2.2.14 OpenSSL/0.9.8l PHP/5.3.1 mod_apreq2-20090110/2.7.1 mod_perl/2.0.4 Perl/v5.10.1)  
3306/tcp  open  mysql    MySQL (unauthorized)  
MAC Address: 08:00:27:7E:40:01 (Oracle VirtualBox virtual NIC)  
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.  
Nmap done: 1 IP address (1 host up) scanned in 14.35 seconds  
  
kali@kali: ~
```

Figure 08: NMAP on DVWA

An `nmap -sV` scan identified five open ports: 21 (FTP – ProFTPD 1.3.2c), 22 (SSH – OpenSSH 5.3p1), 80 (HTTP – Apache 2.2.14/PHP 5.3.1), 443 (HTTPS – same Apache stack, OpenSSL 0.9.8l), and 3306 (MySQL, unauthorized). The host fingerprinted as Linux, consistent with the isolated lab VM.

4.2 Interpretation of Findings

All five services found in `nmap -sV` scan on DVWA (192.168.100.4) are outdated and disclose detailed version banners without authentication, giving an attacker a clear starting point for targeted exploitation. Exposing MySQL directly on the network is unnecessary and widens the attack surface beyond the web application. Port 443 offers TLS, but it runs the same aging OpenSSL build as port 80, so the "encrypted" endpoint isn't inherently trustworthy either. Overall, the scan suggests a system set up once and never patched or hardened afterward.

5. Network Communication Observation

5.1 Captured Traffic

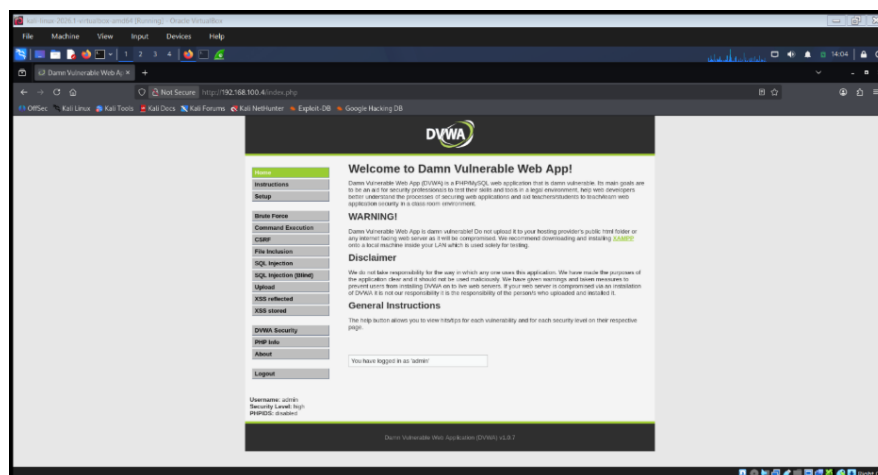


Figure 09: Logged in on DVWA

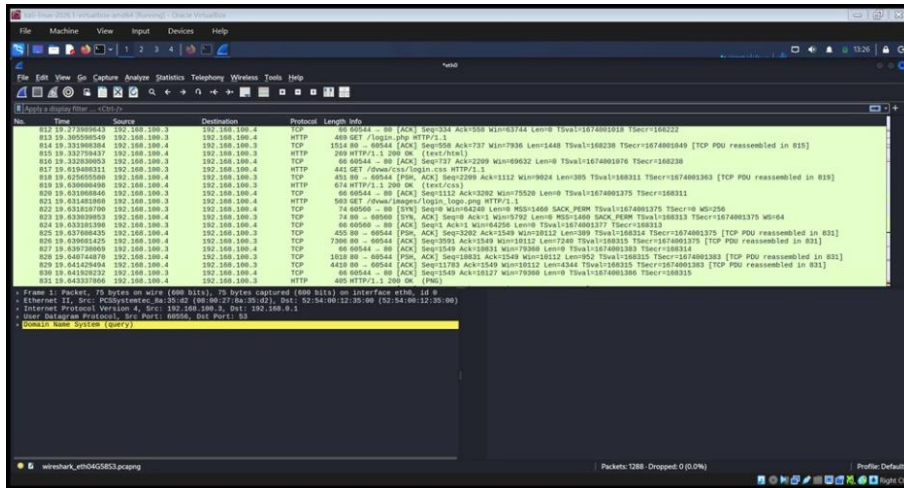


Figure 10: Wireshark capturing activities

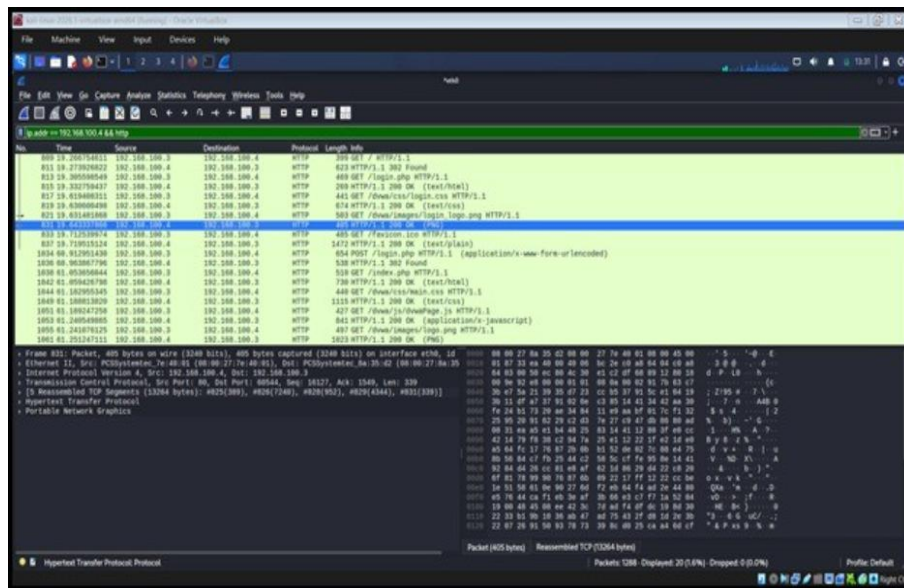


Figure 11: Wireshark capturing filtered by HTTP

Traffic was captured with Wireshark on the Kali interface while filtering on host 192.168.100.3. Logging into DVWA generates an HTTP POST request to /login.php. Following the TCP stream shows the username and password submitted as plaintext form fields (username=..., password=...) inside the unencrypted HTTP body, along with the PHP session cookie issued afterward. Because the connection is plain HTTP rather than HTTPS, any party positioned on the same network segment for example, another host on this NAT network could passively capture these credentials and the session cookie without needing to break any cryptography.

5.2 Sensitive Information Exposed

- Login credentials (username and password) submitted in cleartext over HTTP
- PHPSESSID session cookie, which could be replayed to hijack the session if intercepted
- Server and application banners (Apache/PHP version) visible in HTTP response headers

6. Authentication Strength Evaluation

6.1 Extracting Account and Hash Information

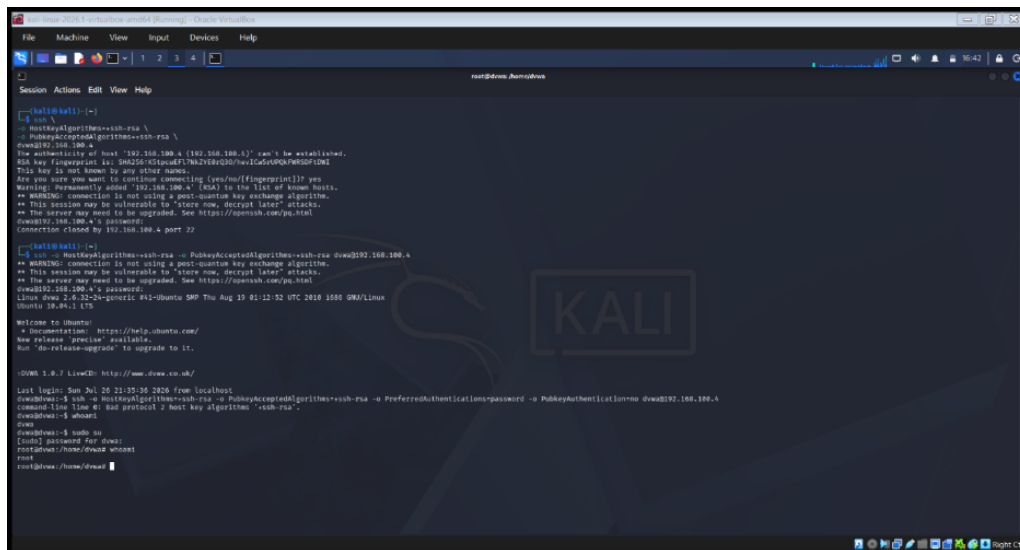


Figure 12: In the root of DVWA

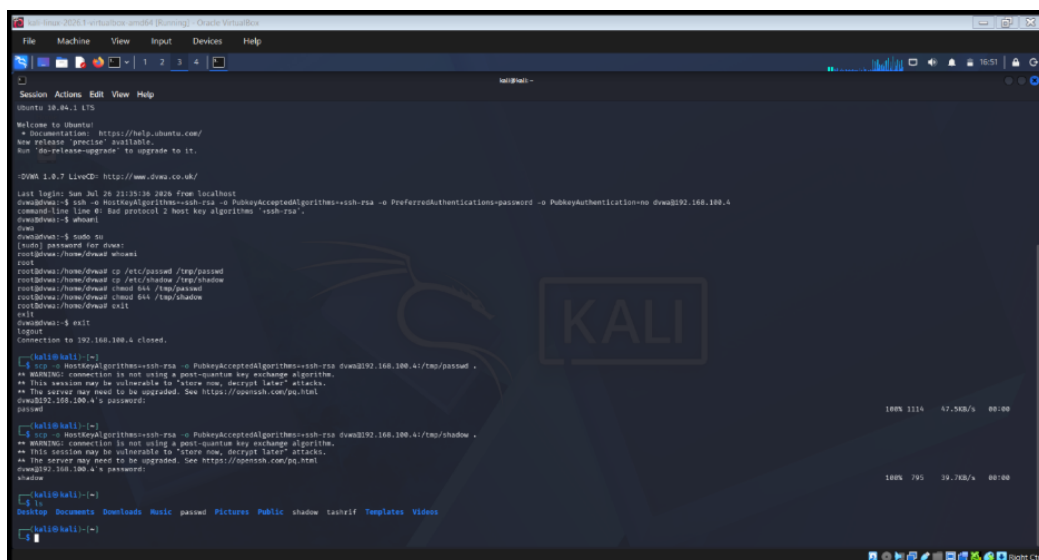
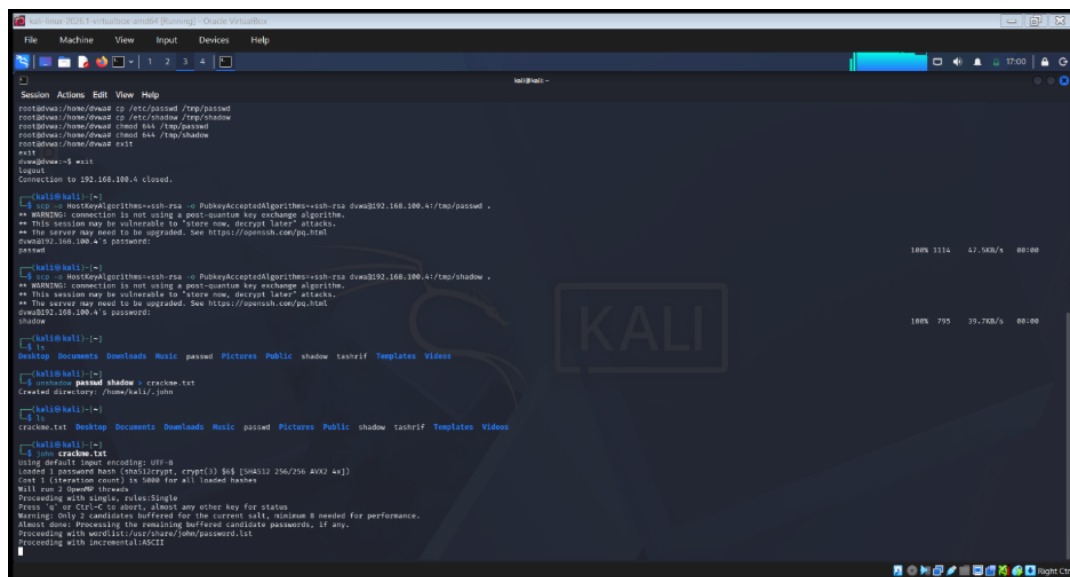


Figure 13: Extracting files from DVWA and copying in Kali Linux

Working from the scenario in which access to the application's stored account data has already been obtained, the 'users' table of the DVWA MySQL database was exported, yielding username/hash pairs such as admin. DVWA stores passwords as unsalted MD5 digests, which is a legacy, cryptographically broken hashing scheme with no per-user salt identical passwords across accounts produce identical hashes, and precomputed rainbow tables are widely available for common values.

6.2 Password Recovery



```
root@kali:~# cp /etc/passwd /tmp/passwd
root@kali:~# cp /etc/shadow /tmp/shadow
root@kali:~# cd /tmp
root@kali:~# pwd
/tmp
root@kali:~# ls
passwd  shadow
root@kali:~# cat passwd
root@kali:~# cat shadow
root@kali:~# john --wordlist=/usr/share/wordlists/rockyou.txt passwd
root@kali:~# john --wordlist=/usr/share/wordlists/rockyou.txt shadow
root@kali:~# cat crack.txt
password
john
```

Figure 14: Using John the Ripper to crack extracted files

The extracted hash file was transferred to the Kali VM, extracted accounts were cracked (/etc/passwd and /etc/shadow) using SSH root privileges and cracked them with John the Ripper. The default DVWA accounts recovered instantly because they use common dictionary words (e.g., 'password', 'admin'), confirming that even when credentials are not stored in plaintext, weak or default password choices make offline recovery trivial. No custom, high-entropy password in the account set would have been recovered from a standard wordlist within a comparable timeframe, illustrating the practical difference password strength makes even under a broken hashing scheme. As the password set as 'emran123' is not being able to recover by John the Ripper.

6.3 Why the Passwords Were Weak

The recovered passwords were short, dictionary-based and reused across accounts. Combined with an unsalted, fast hash algorithm this allowed near-instant offline recovery. Strong hashing alone would have slowed an attacker considerably even with the same weak passwords and strong passwords alone would have resisted the wordlist attack even under MD5 the finding highlights that both password choice and hashing design failed here.

7. Security Analysis and Recommendations

7.1 Identified Weaknesses

- Unencrypted HTTP transport: the login and session mechanism run over plain HTTP, exposing credentials and session cookies to anyone able to observe network traffic between the client and server.
- Weak, unsalted password hashing (MD5) combined with weak default passwords, allowing rapid offline password recovery once the credential store is obtained.
- Unnecessary exposure of the database service (MySQL, port 3306) to the network rather than restricting it to localhost, widening the attack surface beyond the web application itself.
- Outdated/disclosed software versions (Apache, PHP) visible in-service banners, aiding an attacker in fingerprinting known vulnerabilities before any active exploitation.

7.2 Why These Weaknesses Exist

These issues stem from configuration and design choices prioritizing ease of setup over security: DVWA is intentionally shipped in an insecure default state for training purposes, but the same root causes no enforced TLS, legacy hashing left unchanged, services bound to all interfaces by default, and verbose banners are common in real misconfigured deployments where hardening steps are skipped after initial installation.

7.3 Proposed Countermeasures

- Encrypted communication: terminate the site behind HTTPS/TLS (e.g., a reverse proxy with a valid certificate) so credentials and session tokens are never transmitted in clear text.
- Stronger authentication: enforce a minimum password policy (length, complexity, no dictionary words), rate-limit or lock out repeated failed logins and consider multi-factor authentication for administrative accounts.
- Modern password hashing: replace unsalted MD5 with a slow, salted algorithm such as bcrypt or Argon2 and add per-user salts so identical passwords do not yield identical hashes.
- Access control and service hardening: bind MySQL to localhost only (or firewall it from external hosts), run services with least privilege and disable verbose version banners in HTTP responses.
- System updates: keep the web server, PHP runtime and database engine patched to current stable releases to close known, publicly documented vulnerabilities.

8. Task Distribution Among Group Members

Individual responsibilities across the five project phases were as follows:

- Tasnim Tiba (23-54755-3): Environment setup and NAT network configuration (Phase 1) creating the Ubuntu Server target VM in Oracle VirtualBox, deploying DVWA via the vulnerables/web-dvwa Docker image, configuring the isolated MT_Project NAT Network, attaching both the Kali and DVWA VMs to it and verifying connectivity between them. Also responsible for compiling and writing the final report.
- Md. Emran Hossain (23-54780-3): Authentication strength evaluation (Phase 4) extracting the username and password hash pairs from the DVWA, transferring the extracted hashes to the Kali attacker VM, and running the offline cracking process with John the Ripper to recover weak passwords.
- Muhammad Abdur Rahaman Tashrif (23-54785-3): System discovery and network communication observation (Phases 2–3) performing the nmap host discovery and port/service scans against the DVWA target, interpreting open ports and service banners, and capturing and analyzing network traffic with Wireshark to identify the protocol in use and the sensitive information exposed during login.

9. Conclusion

This assessment of DVWA, conducted entirely inside an isolated VirtualBox NAT network, walked through blind service discovery, traffic analysis, and offline credential auditing, and surfaced concrete, remediable weaknesses: unencrypted transport, weak password hashing paired with weak passwords, unnecessary service exposure, and outdated/disclosed software versions. Each finding maps to a standard, well-understood countermeasure (TLS, modern salted hashing, network access control, and patch management), reinforcing that most of the risk observed here comes from configuration choices rather than exotic vulnerabilities the same class of issues that routinely appears in real-world, unhardened deployments.